Rachid Tazi

ASSIGNMENT 1 : REPORT

In this project, I implemented a ray tracer in C++ with several features, including ray-sphere intersection, direct lighting and shadows, mirror reflections, indirect lighting, anti-aliasing, triangle mesh rendering, bounding boxes, and BVH.

LAB 1 : Basic raytracer: spheres, lighting, shadows

We started by building the foundation of the entire ray tracer. The goal is to take a 3D scene and calculate the color of each pixel by casting rays.
For each pixel, we cast a ray, determine which object it hits, calculate the lighting, and apply the color to the image.

Ray-sphere intersection

To determine whether a ray touches a sphere, the program mathematically calculates the intersection between the ray and the sphere to find the point of contact, the distance, and the normal to the surface.



Multiple object intersection

To take multiple objects into account, the program checks for intersections with all of them and keeps only the one closest to the camera.

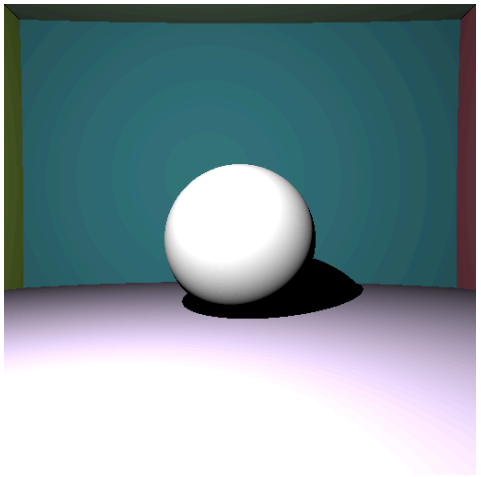
Lambertian direct lighting

This feature calculates the direct light coming from the light source, and the intensity depends on the surface's orientation and its distance from the light.

Shadows

To determine whether a point is in shadow, the program sends a ray toward the light source. If an object blocks this ray, the point is considered to be in shadow. A very small offset is added to the starting points of the secondary rays to avoid numerical errors.

Here is the result obtained at the end of this first lab:



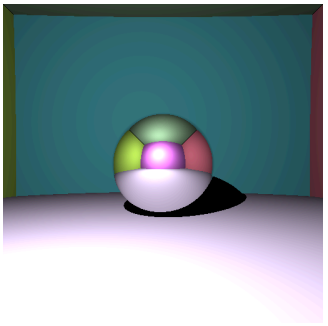
LAB 2: Recursive raytracing

Recursive path tracing

The program uses a recursive function to track light bounces in the scene. When a ray hits a surface, new rays are generated to simulate reflections and indirect lighting.

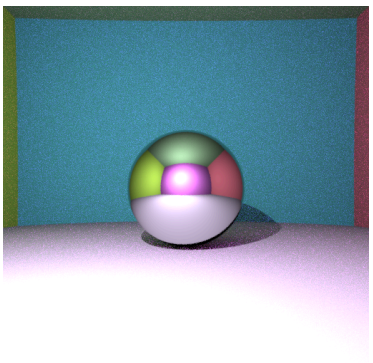
Mirror reflections

Mirror surfaces reflect light rays according to the laws of reflection.



Indirect lighting

This section simulates light that bounces off objects before reaching a surface. This allows us to obtain areas that are indirectly lighted.



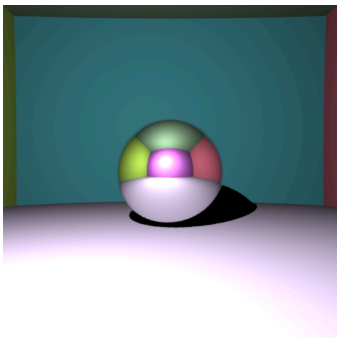
Monte Carlo multi-sampling & Antialiasing

- Monte Carlo

Multiple rays are cast for each pixel and then averaged together. This reduces the noise in the image.

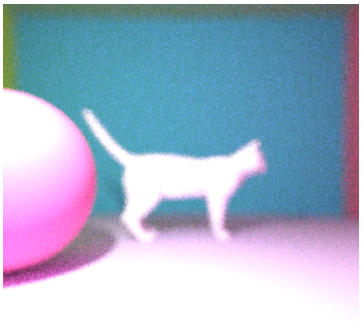
- Antialiasing

The rays are randomly shifted slightly within each pixel to reduce the visible aliasing along the edges of objects.



Depth of Field

This feature simulates a real camera with an aperture. Objects that are far from the focal plane appear blurry.

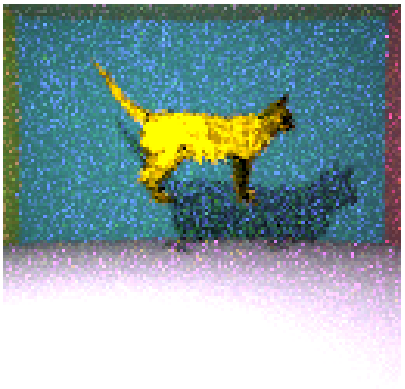


Lab 3 : Triangle Meshes

In this lab, we load .obj 3D models to work with complex objects, such as a cat. The objects are represented as triangles.

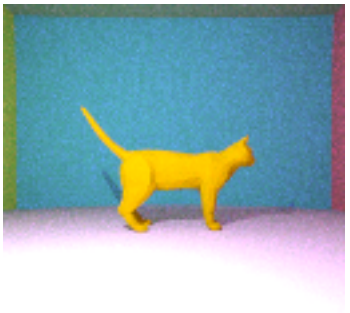
Ray-triangle intersection

This section allows us to test for the intersection between a ray and a triangle. This will enable the meshes to be displayed.
The barycentric coordinates are used to verify whether the intersection point is indeed inside the triangle.



Mesh scaling and translation

3D models can be moved and resized within the scene to position objects correctly in the environment.



Bounding Box

An bounding box is calculated around the 3D model. Before testing all the triangles for a given ray, the program checks whether that ray intersects the bounding box, which significantly speeds up rendering.

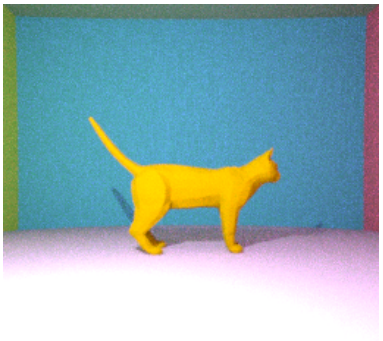
Lab 4 : BVH acceleration

BVH structure

The Bounding Volume Hierarchy organizes triangles into a tree-like structure composed of nested bounding boxes. This further speeds up intersection calculations. The BVH is constructed recursively by dividing the triangles into several smaller groups.

The triangles are separated along the longest axis of the bounding box to improve the quality of the BVH, and the triangles are distributed within the BVH based on the position of their barycenter.

Here is the final result:



ANALYSIS OF TIMINGS

To analyze the effectiveness of the methods, I use render times. For each method, I run two tests (a quick setup and a final render).

- Fast setup

```
int W = 128;
int H = 128;
scene.nb_paths = 16;
scene.max_path_length = 3;
```

- Final render

```
int W = 512;
int H = 512;
scene.nb_paths = 32;
scene.max_path_length = 5;
```

NAIVE MESH RAY TRACER

fast setup

```
 real    0m19.307s
user    6m7.832s
sys     0m0.091s
```

final render

```
real    12m43.234s
user    251m15.132s
sys     0m2.497s
```

BOUNDING BOX

fast setup
real 0m4.942s
user 1m29.567s
sys 0m0.080s

final render
real 2m36.957s
user 51m34.705s
sys 0m0.368s

BVH version

fast setup
real 0m0.196s
user 0m3.399s
sys 0m0.021s

final render
real 0m5.860s
user 1m54.920s
sys 0m0.037s

Thus, we can see that the BVH performs the best.